**Azərbaycan Respublikası Elm və Təhsil Nazirliyi**

**Qarabağ Universiteti**

**Mühəndislik Fakültəsi**

## Verilənlərin Strukturu və Alqoritmlər

*Mexatronika və robototexnika mühəndisliyi və Kompüter mühəndisliyi ixtisası üzrə bakalavr proqramının tələbələri üçün.*

# Laboratoriya işi № 2

**Tələbənin adı və soyadı: Muhəmməd Abdullayev**

**Qrup: KOM25A**

**Müəllim: Şamil Hümbətov**

**Tarix: 19 oktyabr 2026**

**2026**

## Giriş

Bu işdə Linked List, Stack və Queue strukturları Python-da sıfırdan yazılıb və 100, 1000, 10000 element üzərində performans testləri aparılıb. Əsas mənbə: M. A. Weiss, Data Structures and Algorithm Analysis in C++ (4th Edition).

**Tapşırıq 1 – Playlist (Linked List): src/playlist.py**
**Tapşırıq 2 – Undo History (Stack): src/undo_stack.py**
**Tapşırıq 3 – Print Queue (Queue): src/print_queue.py**

Testlər src/benchmark.py ilə aparılıb, nəticələr test_results/ qovluğundadır. Hazır konteynerlər (collections.deque, queue.*) istifadə edilməyib; bütün strukturlar öz Node siniflərilə yazılıb.

## **Tapşırıq 1.** Smart Music Playlist (Linked List)

### A. Araşdırma

Array-də elementlər ardıcıl yerləşir, indekslə müraciət O(1)-dir, əlavə və silmə isə sürüşdürmə tələb edir (O(n)). Linked List-də düyünlər göstəricilərlə bağlıdır, müraciət O(n)-dir, düyün tapıldıqdan sonra əlavə və silmə göstəricilərin dəyişdirilməsi ilə aparılır.

Playlistdə mahnılar tez-tez əlavə və silinir, ölçü əvvəlcədən məlum olmur, çalma isə ardıcıldır. Linked List bunlara uyğundur.

Singly: bir göstərici, az yaddaş, yalnız irəli keçid. Doubly: iki göstərici, hər iki istiqamət və düyünün O(1) silinməsi, lakin daha çox yaddaş.

Düyünlər yaddaşda ardıcıl deyil, indekslə keçid yoxdur və keşdən pis istifadə olunur. Axtarış O(n)-dir.

Linked List: mövqe tapıldıqdan sonra yalnız göstəricilər dəyişir. Doubly Linked List daha əlverişlidir.

### B. Proqramlaşdırma

SongNode – Song ID, Name, Artist, Duration, next.

Playlist – head, tail, size. Funksiyalar: addSong(), deleteSong(), searchSong(), displayPlaylist(), countSongs(), playNextSong().

addSong() tail sayəsində O(1)-dir; qalan əməliyyatlar O(n)-dir. Menyu: Add, Delete, Search, Display, Play Next, Count, Exit.

**Şəkil: screenshots/task1_menu.png**

### C. Təcrübə və Eksperiment

Insert – N əlavənin ümumi vaxtı. Search və Delete – ən pis hal (son element), bir əməliyyatın orta vaxtı. Hər ölçmə 5 dəfə təkrarlanıb, median götürülüb (time.perf_counter). Nəticələr Cədvəl 1 və Şəkil 1-dədir.

## **Tapşırıq 2.** Browser Undo History (Stack)

### A. Araşdırma

LIFO: ən son daxil edilən element ilk çıxarılır.

Back düyməsi ən son baxılan səhifədən əvvəlkinə qaytarır, bu isə pop əməliyyatıdır.

Hər əməliyyat push edilir; Undo zamanı top element pop olunub geri qaytarılır.

Queue FIFO olduğu üçün ən köhnə əməliyyat geri qaytarılardı, bu isə Undo məntiqini pozardı.

Linked List: ölçü məhduddur, hər düyün əlavə yaddaş tələb edir. Array: kompaktdır və sürətlidir, lakin tutum dolanda yenidən ayırma lazımdır.

### B. Proqramlaşdırma

ActionNode – Action ID, Name, Time, next.

UndoStack – top. Funksiyalar: pushAction(), undoAction(), currentAction(), displayHistory(), countActions().

Push və pop top-da aparılır, hər ikisi O(1)-dir. Menyu: Perform, Undo, View Current, Display History, Exit.

**Şəkil: screenshots/task2_menu.png**

### C. Təcrübə və Eksperiment

100, 1000 və 10000 Push və Pop əməliyyatının ümumi vaxtı ölçülüb (5 təkrar, median). Pop-dan əvvəl Stack N elementlə doldurulub. Nəticələr Cədvəl 2 və Şəkil 2-dədir.

## **Tapşırıq 3.** University Printing Service (Queue)

### A. Araşdırma

FIFO: ilk daxil olan element ilk xidmət alır.

Stack-də əlavə və çıxarma eyni ucdadır (LIFO), Queue-da isə rear-də əlavə, front-da çıxarmadır (FIFO).

Növbədə əvvəl gələn əvvəl xidmət almalıdır; Queue bunu təmin edir.

Queue olmasa ardıcıllıq pozulur, əvvəl gələnlər uzun gözləyə bilər.

Circular Queue boşalmış yerləri təkrar istifadə edir və elementləri sürüşdürməyi tələb etmir.

**B. Proqramlaşdırma**

PrintJobNode – Job ID, Student Name, Pages, next.

PrintQueue – front, rear. Funksiyalar: enqueue(), dequeue(), peek(), displayQueue(), countJobs().

Bütün əsas əməliyyatlar O(1)-dir. Menyu: Add Job, Print Next, View Next, Display, Count, Exit.

**Şəkil: screenshots/task3_menu.png**

**C. Təcrübə və Eksperiment**

100, 1000 və 10000 elementli növbələrdə Enqueue, Dequeue və Peek üçün N əməliyyatın ümumi vaxtı ölçülüb (5 təkrar, median). Nəticələr Cədvəl 3 və Şəkil 3-dədir.

**Test nəticələri**

Hər proqramın menyusundakı bütün əməliyyatlar yoxlanılıb və gözlənilən nəticələri verib. Performans testlərinin nəticələri 6-cı bölmədə verilib.

**Şəkil: screenshots/task1_test.png**
**Şəkil: screenshots/task2_test.png**
**Şəkil: screenshots/task3_test.png**
**Şəkil: screenshots/benchmark_output.png**

Sistem məlumatları:

| Parametr | Dəyər |
|---|---|
| Computer Model | Lenovo LOQ 15 |
| CPU | Intel Core i7-14700HX (28 məntiqi nüvə) |
| RAM | 16 GB |
| GPU | NVIDIA GeForce RTX 4060 |
| Operating System | Windows 11 (10.0.26100) |
| Compiler / Interpreter | Python (CPython) |
| Compiler Version | 3.14.3 |
| IDE | Visual Studio Code |
| Screen Resolution | 1920x1080 |
| Storage Type (SSD/HDD) | SSD |
| Power Mode | Performance (şarj cihazına qoşulu) |
| Other Factors | Python tək nüvədən istifadə edir; ölçmələr 5 təkrarın medianıdır |

## Cədvəllər

**Cədvəl 1.** Playlist (Linked List)

| Element sayı | Insert (ms) | Search (ms) | Delete (ms) |
|---|---|---|---|
| 100 | 0.0408 | 0.003338 | 0.0044 |
| 1000 | 0.4033 | 0.028794 | 0.0318 |
| 10000 | 2.515 | 0.21886 | 0.20215 |

Insert – N əməliyyatın ümumi vaxtı; Search və Delete – ən pis hal, bir əməliyyatın orta vaxtı.

**Cədvəl 2.** Stack

| Əməliyyat sayı | Push (ms) | Pop (ms) |
|---|---|---|
| 100 | 0.1217 | 0.0073 |
| 1000 | 1.223 | 0.079 |
| 10000 | 13.2669 | 0.7236 |

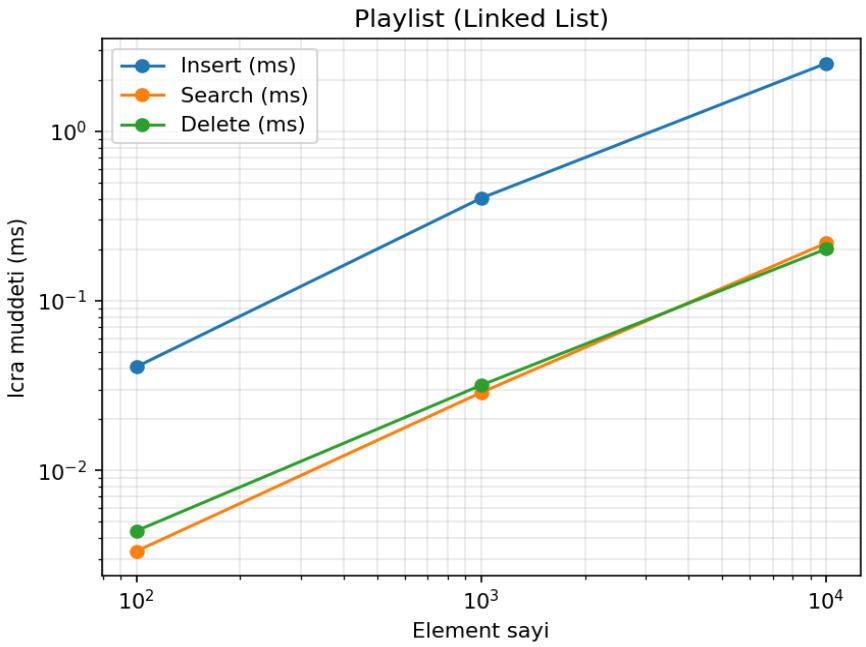
**Cədvəl 3.** Queue

| Element sayı | Enqueue (ms) | Dequeue (ms) | Peek (ms) |
|---|---|---|---|
| 100 | 0.0129 | 0.007 | 0.0028 |
| 1000 | 0.126 | 0.0688 | 0.0225 |
| 10000 | 1.8513 | 0.7241 | 0.2444 |

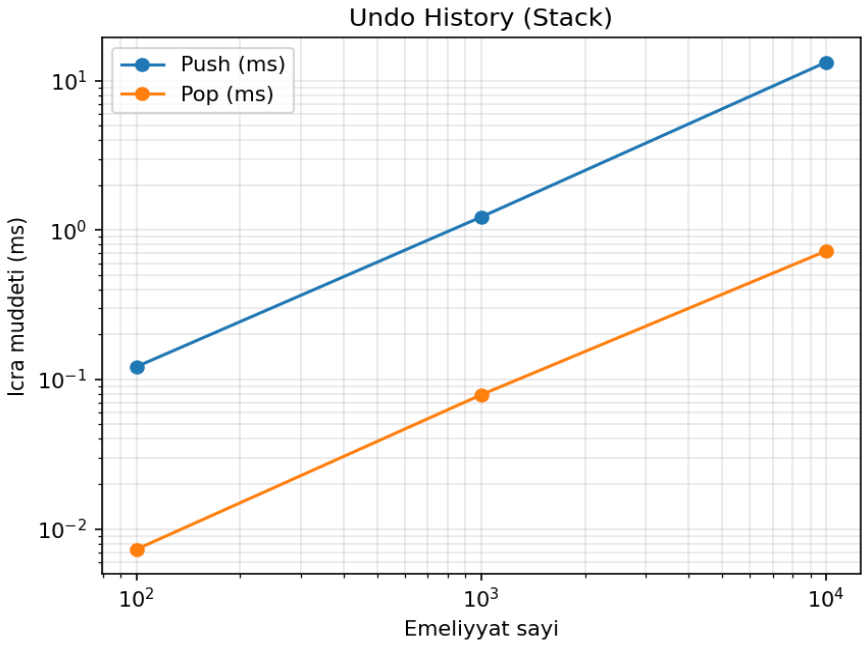
Stack və Queue üçün: N əməliyyatın ümumi vaxtı.

## Qrafiklər

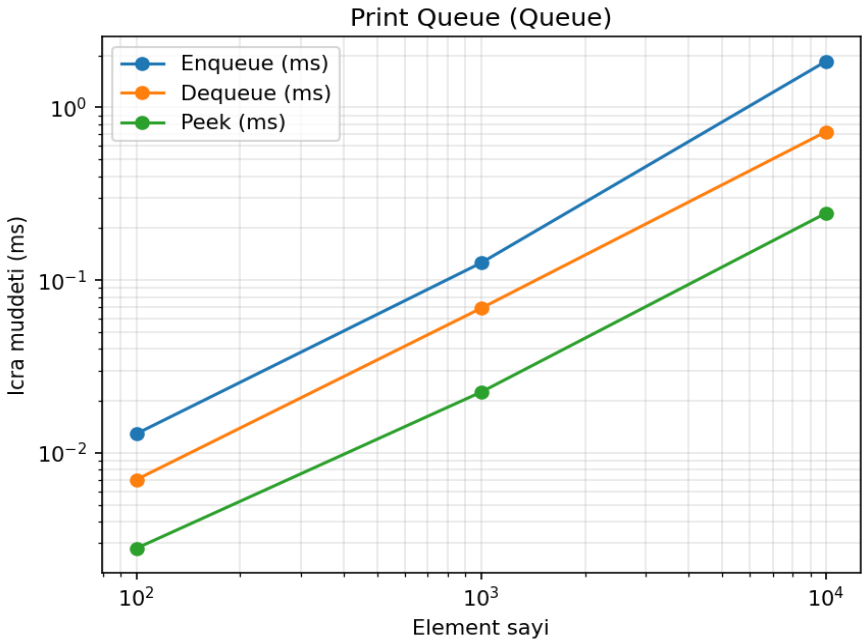
X oxu element (əməliyyat) sayı, Y oxu icra müddətidir (ms). Hər iki ox loqarifmikdir.




**Şəkil 1.** Playlist (Linked List)




**Şəkil 2.** Stack: Push və Pop




**Şəkil 3.** Queue

## Analiz

### Tapşırıq 1

N 100-dən 10000-ə qədər artdıqda Search 0.0033 ms-dən 0.219 ms-ə (təxminən 66 dəfə), Delete 0.0044 ms-dən 0.202 ms-ə (təxminən 46 dəfə) artıb. Bu əməliyyatlar siyahını gəzdiyi üçün ən çox təsirlənib. Insert-in ümumi vaxtı 0.0408 ms-dən 2.515 ms-ə qədər artıb (təxminən 62 dəfə); bir əlavənin vaxtı isə təxminən sabit qalıb (0.0004 ms-ə yaxın).

Bəli, təxminən. N 10 dəfə artdıqda Search 8.6 və 7.6 dəfə, Delete 7.2 və 6.4 dəfə artıb, yəni O(n) xətti artıma yaxındır. Artımın 10-dan bir qədər az olması sabit xərclərin (funksiya çağırışı, interpretator) kiçik N-də payının böyük olması ilə izah oluna bilər.

Array-də (Python list) indekslə müraciət O(1)-dir, ona görə indeksə görə axtarış Linked List-dən sürətli olardı. Dəyərə görə axtarış hər ikisində O(n)-dir, lakin array keşdən daha yaxşı istifadə etdiyi üçün adətən daha sürətlidir. Ortadan silmə array-də sürüşdürməyə görə O(n)-dir.

Əsas üstünlük: düyün tapıldıqdan sonra əlavə və silmənin sürüşdürmə olmadan, göstəricilərin dəyişdirilməsi ilə aparılması və ölçünün dinamik olması.

Bəli. Əvvəlki mahnıya qayıtmaq və düyünü əvvəlki düyünü axtarmadan silmək üçün Doubly Linked List daha məntiqlidir. Əvəzində hər düyündə əlavə göstərici yaddaşı tələb olunur.

### Tapşırıq 2

Pop həmişə Push-dan sürətli olub: N=10000 üçün Push 13.27 ms, Pop 0.72 ms (təxminən 18 dəfə). Push hər dəfə yeni ActionNode yaradır və cari vaxtı mətnə çevirir (datetime.now().strftime), Pop isə yalnız top göstəricisini dəyişir.

Bəli. N 10 dəfə artdıqda Push təxminən 10.1 və 10.9 dəfə, Pop 10.8 və 9.2 dəfə artıb. Ümumi vaxtın N ilə xətti artması bir əməliyyatın O(1) olduğunu göstərir.

Array əsaslı Stack-də düyün yaradılmır, yaddaş kompakt olur və keş daha yaxşı işləyir; yalnız tutum dolanda yenidən ayırma xərci yaranır. Ona görə Push adətən daha sürətli olardı.

Geri qayıtma həmişə ən son səhifəyə doğru baş verir, bu isə LIFO prinsipi ilə üst-üstə düşür.

Call stack, mötərizələrin balansının yoxlanması, postfix ifadələrin hesablanması, DFS alqoritmi.

**Tapşırıq 3**

Elementlər daxil olma ardıcıllığı ilə çıxarılır: ilk əlavə edilən ilk xidmət alır (məsələn Ali, sonra Leyla, sonra Nigar).

Bəli. N 10 dəfə artdıqda Dequeue 9.8 və 10.5 dəfə, Peek 8.0 və 10.9 dəfə artıb, yəni ümumi vaxt N ilə xətti artır. Enqueue 9.8 və 14.7 dəfə artıb; 10000-də artımın bir qədər çox olması yaddaş ayırması və zibil yığıcı ilə əlaqəli ola bilər. Əməliyyatların sıralanması: Peek ən sürətli (0.244 ms), Dequeue (0.724 ms), Enqueue ən yavaş (1.851 ms), çünki Enqueue yeni düyün yaradır.

Stack istifadə olunsaydı sonuncu gələn sərnişin ilk xidmət alardı, bu ədalətsizdir.

Bir əməliyyatın vaxtı təxminən sabit qalıb, ümumi vaxt isə N ilə artıb.

Boşalmış yerlərin təkrar istifadəsi, elementlərin sürüşdürülməməsi və sabit yaddaş istifadəsi.

Sistem parametrlərinin təsiri

CPU tezliyi və keş vaxtları birbaşa təyin edir; Linked List düyünləri səpələndiyi üçün keşə həssasdır.

Arxa plan prosesləri və enerji qənaəti rejimi vaxtları artıra bilər; buna görə median götürülüb.

Python interpretator olduğu üçün C++ ilə müqayisədə hər əməliyyat daha bahalıdır.

Kompüterdə 28 məntiqi nüvə olsa da, Python kodu (GIL) bir nüvədə işləyir, ona görə nüvə sayı nəticələri sürətləndirmir.

RAM, GPU, Storage və ekran ayırdetməsinin təsiri bu testlərdə minimaldır, çünki fayl və qrafik əməliyyatı ölçülmür.

## Nəticə

Linked List-də axtarış və silmə O(n), tail ilə əlavə O(1)-dir. Stack və Queue-da bütün əsas əməliyyatlar O(1)-dir. Ölçmələr bunu təsdiq edib: N 10 dəfə artdıqda vaxt təxminən 10 dəfə artıb. Push Pop-dan, Enqueue Dequeue və Peek-dən yavaş olub, çünki yeni düyün yaradır. Diqqət tələb edən məqamlar: ilk düyün silinəndə head, son düyün silinəndə tail və rear göstəricilərinin yenilənməsi.

## İstinadlar

Weiss, M. A. Data Structures and Algorithm Analysis in C++, 4th Edition. Pearson, 2014.

Goodrich, M. T., Tamassia, R., Goldwasser, M. H. Data Structures and Algorithms in Python. Wiley, 2013.

Python documentation, time.perf_counter: https://docs.python.org/3/library/time.html

**AI Usage Declaration**

**Tool: Claude AI**

**Purpose: Kodların yazılması və araşdırmalardan sonra məlumatların toplu halda yazılması**

**Used Sections: Hesabatın mətn hissəsi (Giriş, Araşdırma, Analiz, Nəticə). Kodlar tələbə tərəfindən sıfırdan yazılıb.**

**Mövzu: Linked List, Stack və Queue Verilənlər Strukturlarının Tətbiqi, Təhlili və Müqayisəsi**

**Əsas mənbə:** *Mark Allen Weiss, Data Structures and Algorithm Analysis in C++ (4th Edition)*

## Məqsəd

Bu laboratoriya işinin məqsədi tələbələrin Linked List, Stack və Queue verilənlər strukturlarının nəzəri əsaslarını öyrənməsi, onları sıfırdan C++ / Python dilində implementasiya etməsi, eksperimental sınaqlar aparması və nəticələri elmi əsaslarla təhlil etməsidir.

## Öyrənmə Nəticələri

Laboratoriya işini uğurla tamamlayan tələbə:

- Linked List, Stack və Queue strukturlarının iş prinsiplərini izah edə bilir;
- Verilənlər strukturlarını STL konteynerlərindən istifadə etmədən implementasiya edə bilir;
- Real həyat problemlərini uyğun verilənlər strukturları vasitəsilə modelləşdirə bilir;
- Eksperimental nəticələri cədvəl və qrafiklər vasitəsilə təqdim edə bilir;
- Teorik və praktik mürəkkəblikləri müqayisə edə bilir;
- Performansa təsir edən sistem parametrlərini müəyyən edə bilir.

## Tapşırıq 1

## Smart Music Playlist (Linked List)

## Real Həyat Ssenarisi

Musiqi tətbiqində (Spotify, Apple Music və s.) istifadəçinin playlisti saxlanılır. İstifadəçi mahnı əlavə edə, silə və playlisti görüntüləyə bilir.

## A. Araşdırma

Aşağıdakı sualları cavablandırın:

1. Array və Linked List arasında əsas fərqlər hansılardır?
2. Niyə musiqi playlistləri üçün Linked List uyğun seçim hesab oluna bilər?
3. Singly və Doubly Linked List arasında hansı üstünlüklər və çatışmazlıqlar mövcuddur?
4. Linked List-də axtarış niyə Array-dən daha yavaş ola bilər?

5. Playlistdə mahnıların sırasını dəyişmək üçün hansı struktur daha əlverişlidir?

**B. Proqramlaşdırma**

Aşağıdakı funksiyaları dəstəkləyən Playlist sistemi hazırlayın:

**Song**

- Song ID
- Song Name
- Artist

**Əməliyyatlar**
Add Song

Delete Song

Search Song

Display Playlist

Count Songs

**Məhdudiyyət**

Hazır STL list istifadəsi qadağandır.

**C. Təcrübə və Eksperiment**

Playlist sistemini aşağıdakı ölçülərdə test edin:

100 mahnı

1000 mahnı

10000 mahnı

Aşağıdakı əməliyyatların vaxtını ölçün:

Insert

Search

Delete

**Nəticələri aşağıdakı formada təqdim edin**

| Element sayı | Insert (ms) | Search (ms) | Delete (ms) |
|---|---|---|---|
| **100** | | | |
| **1000** | | | |
| **10000** | | | |

**D. Analiz və Nəticələrin Şərhi**

Ən azı aşağıdakı sualları cavablandırın:

1. Element sayı artdıqca hansı əməliyyat daha çox təsirləndi?
2. Təcrübə nəticələri Big-O nəzəriyyəsini təsdiqləyirmi?
3. Eyni problemi Array istifadə etməklə həll etsəydiniz nəticələr necə olardı?
4. Linked List-in əsas üstünlüyü nədir?
5. Playlist tətbiqində Doubly Linked List istifadə etmək daha məntiqli olardımı?

**Qrafik**

Qrafikdə göstərilməlidir:

- X oxu → Element sayı
- Y oxu → İcra müddəti

**Tapşırıq 2**

**Browser History Manager (Stack)**

**Real Həyat Ssenarisi**

Veb brauzerdə istifadəçi müxtəlif səhifələrə keçid edir. "Back" düyməsi basıldıqda əvvəlki səhifələrə qayıdır.

Bu mexanizm Stack prinsipi ilə işləyir.

**A. Araşdırma**

1. Stack strukturunda LIFO prinsipi nədir?
2. Browser History sistemi nə üçün Stack istifadə edir?
3. Undo əməliyyatı necə Stack vasitəsilə həyata keçirilir?
4. Queue istifadə olunsaydı hansı problemlər yaranardı?
5. Linked List və Array əsaslı Stack implementasiyalarını müqayisə edin.

**B. Proqramlaşdırma**

Aşağıdakı menyunu hazırlayın:

Visit Page

Back

Current Page

Display History

Exit

**Məlumat**
Page ID

URL

Visit Time

**Məhdudiyyət**

Hazır STL stack istifadəsi qadağandır.

**C. Təcrübə və Eksperiment**

Aşağıdakı həcmdə əməliyyatlar yaradın:

100 Push/Pop

1000 Push/Pop

10000 Push/Pop

Ölçün:

Push müddəti

Pop müddəti

**Cədvəl**

| Əməliyyat sayı | Push (ms) | Pop (ms) |
|---|---|---|
| 100 | | |
| 1000 | | |
| 10000 | | |

**D. Analiz və Nəticələrin Şərhi**

1. Push və Pop əməliyyatları arasında fərq müşahidə olundumu?
2. Əldə olunan nəticələr O(1) mürəkkəbliyini dəstəkləyirmi?
3. Array əsaslı Stack ilə müqayisədə hansı fərqlər yarana bilər?
4. Brauzerlərdə tarixçənin saxlanması üçün niyə Stack istifadə olunur?
5. Stack-in real sistemlərdə başqa hansı tətbiqləri mövcuddur?

**Qrafik**

Push və Pop əməliyyatlarının performans müqayisəsini təqdim edin.

**Tapşırıq 3**

**Airport Check-in Queue System (Queue)**

**Real Həyat Ssenarisi**

Hava limanında sərnişinlər qeydiyyatdan keçmək üçün növbəyə dayanırlar.

Birinci gələn sərnişin birinci xidmət alır (FIFO).

## A. Araşdırma

1. Queue strukturunun FIFO prinsipi nədir?
2. Queue və Stack arasında əsas fərqlər hansılardır?
3. Hava limanında növbə idarəetməsi üçün Queue niyə uyğundur?
4. Queue istifadə olunmasa hansı problemlər yarana bilər?
5. Circular Queue nə kimi üstünlüklər təqdim edir?

## B. Proqramlaşdırma

Aşağıdakı sistemi yaradın:

**Passenger**
Passenger ID

Name

Flight Number

**Əməliyyatlar**
Add Passenger

Check-In Passenger

Peek Next Passenger

Display Queue

Count Passengers

## C. Təcrübə və Eksperiment

Süni olaraq aşağıdakı ölçülərdə növbələr yaradın:

100 sərnişin

1000 sərnişin

10000 sərnişin

Ölçün:

Enqueue

Dequeue

Peek

**Cədvəl**

| Element sayı | Enqueue (ms) | Dequeue (ms) | Peek (ms) |
|---|---|---|---|
| 100 | | | |
| 1000 | | | |
| 10000 | | | |

## D. Analiz və Nəticələrin Şərhi

1. FIFO prinsipi praktiki nəticələrdə necə müşahidə olunur?
2. Queue əməliyyatlarının mürəkkəbliyi nəzəri nəticələrə uyğundurmu?
3. Queue əvəzinə Stack istifadə olunsa nə baş verərdi?
4. Növbə ölçüsü artdıqca hansı dəyişikliklər müşahidə olundu?
5. Circular Queue istifadə etməyin üstünlükləri nələrdir?

## Qrafik

Element sayı ilə əməliyyat vaxtları arasındakı əlaqəni qrafiklə göstərin.

## Sistem Məlumatları (Məcburidir)

Bütün analizlərdə aşağıdakı məlumatlar göstərilməlidir:

Computer Model:

CPU:

RAM:

GPU:

Operating System:

Compiler:

Compiler Version:

IDE:

Screen Resolution:

Storage Type (SSD/HDD):

Power Mode:

Other Factors:

**Nümunə**
Computer Model: Lenovo ThinkPad E14

CPU: Intel Core i5-1235U

RAM: 16 GB

GPU: Intel Iris Xe

OS: Windows 11 Pro

Compiler: GCC

Version: 13.2

Resolution: 1920x1080

Storage: NVMe SSD

Analiz zamanı bu parametrlərin nəticələrə necə təsir edə biləcəyi izah edilməlidir.

**Hesabat Strukturı**

Tələbə PDF formatında hesabat təqdim etməlidir.

**Hesabat bölmələri**

1. Giriş
2. Tapşırıq 1
3. Tapşırıq 2
4. Tapşırıq 3
5. Test nəticələri
6. Cədvəllər
7. Qrafiklər
8. Analiz
9. Nəticə
10. İstinadlar

**GitHub Tələbləri**

Repository aşağıdakı struktura malik olmalıdır:

Lab02/

│

├── src/

├── include/

├── screenshots/

├── report/

│ └── report.pdf

├── test_results/

├── README.md

└── docs/

**Minimum tələblər**

- Minimum 5 meaningful commit
- README faylı
- Tam kompilyasiya olunan kod
- Screenshotlar
- Hesabat
- Test nəticələri

**Süni İntellektdən İstifadə Siyasəti**

Bu laboratoriya işində AI alətlərindən (ChatGPT, Copilot, Gemini və s.) istifadə yalnız aşağıdakı məqsədlərlə icazəlidir:

- Sintaksis izahı
- Compiler xətalarının izahı
- C++ və Python dil konstruksiyalarının öyrənilməsi

AI vasitəsilə yaradılmış kodun birbaşa təqdim edilməsi qadağandır.

Tələbə istifadə etdiyi bütün AI alətlərini hesabatın sonunda ayrıca qeyd etməlidir:

AI Usage Declaration

Tool:

Purpose:

Used Sections:

AI istifadəsinin bəyan edilməməsi akademik pozuntu hesab ediləcək.

## Akademik Dürüstlük Qaydaları

Qadağandır:

- Başqa tələbənin kodunu köçürmək
- GitHub repozitoriyalarından hazır həll təqdim etmək
- Süni intellekt vasitəsilə yaradılmış kodu müəllif kimi təqdim etmək
- Hesabatın plagiat formada hazırlanması

Akademik pozuntu aşkar olunarsa iş qiymətləndirilmədən geri qaytarıla bilər.

## Qiymətləndirmə Meyarları (100 bal)

| Meyar | Bal |
|---|---|
| Araşdırma hissəsi | 15 |
| Kod keyfiyyəti və OOP yanaşması | 15 |
| Düzgün implementasiya | 20 |
| Eksperiment və testlər | 15 |
| Analiz və nəticələrin şərhi | 15 |
| Cədvəllər və qrafiklər | 10 |
| Hesabatın keyfiyyəti | 5 |
| GitHub fəaliyyəti və commitlər | 5 |
| **Cəmi** | **100** |

## Təhvil Verilən Fayllar

- GitHub Repository Linki
- PDF Hesabat (yazı qaydaları və nümunə nəzərə alın, BlackBoard yüklənməlidir.)
- Mənbə Kodları
- Test Nəticələri
- Qrafiklər və Screenshotlar

**Qeyd:** Bu laboratoriya işi yalnız proqramlaşdırma bacarığını deyil, həm də tələbənin verilənlər strukturlarını araşdırmaq, eksperimental şəkildə qiymətləndirmək və nəticələri elmi əsaslandırmaq qabiliyyətini ölçür. Bu yanaşma beynəlxalq universitetlərdə tətbiq edilən araşdırma və analiz yönümlü Data Structures laboratoriyalarına uyğun hazırlanmışdır.

# Hər variantda proqramlaşdırma hissələrinin izahı.

**Tapşırıq 1: Spotify Playlist Manager (Linked List)**

**B. Proqramlaşdırma**

Bu tapşırıqda siz musiqi playlistini idarə edən sadə bir proqram hazırlayacaqsınız.

Linked List strukturu vasitəsilə mahnılar bir-birinə bağlanmalıdır. Hər mahnı ayrıca bir node (qovşaq) kimi saxlanmalıdır.

**Yaradılacaq siniflər**

***SongNode***

Aşağıdakı məlumatları saxlamalıdır:

Song ID

Song Name

Artist

Duration

Pointer to Next Song

***Playlist***

Playlist sinfi aşağıdakı funksiyaları təmin etməlidir:

addSong()

deleteSong()

searchSong()

displayPlaylist()

countSongs()

playNextSong()

**Proqramın işləmə ssenarisi**

İstifadəçi menyudan seçim etməlidir:

1. Add Song

2. Delete Song

3. Search Song

4. Display Playlist

5. Play Next Song

6. Count Songs

7. Exit

**Nümunə**
Add Song

ID: 1

Name: Believer

Artist: Imagine Dragons

Song added successfully.

**Tələbə öyrənəcək**

- Linked List node-larının yaradılması
- Pointer istifadəsi
- Traversal
- İstənilən elementin əlavə edilməsi və silinməsi
- Dinamik yaddaş idarəetməsi

---

**Tapşırıq 2: Browser Undo History (Stack)**

**B. Proqramlaşdırma**

Bu tapşırıqda mətn redaktoru və ya brauzerdəki Undo funksiyasını modelləşdirəcəksiniz.

Stack prinsipi:

Last In → First Out (LIFO)

ən son edilən əməliyyat ilk geri qaytarılır.

**Yaradılacaq siniflər**

***ActionNode***

Saxlanılacaq məlumatlar:

Action ID

Action Name

Time

Pointer to Next Action

***UndoStack***

Aşağıdakı funksiyaları yaratmalısınız:

pushAction()

undoAction()

currentAction()

displayHistory()

countActions()

**Proqramın işləmə ssenarisi**

İstifadəçi aşağıdakı əməliyyatları edə bilməlidir:

Perform Action

Undo Action

View Current Action

Display Full History

Exit

**Nümunə**

Perform Action: Write Paragraph

Perform Action: Insert Image

Perform Action: Change Font

Stack vəziyyəti:

TOP

Change Font

Insert Image

Write Paragraph

İstifadəçi Undo etdikdə:

Undo: Change Font

və həmin element Stack-dən çıxarılmalıdır.

**Tələbə öyrənəcək**

- Stack ADT
- Push və Pop əməliyyatları
- Top anlayışı
- LIFO prinsipi
- Undo sistemlərinin işləmə prinsipi

**Tapşırıq 3: University Printing Service (Queue)**

**B. Proqramlaşdırma**

Bu tapşırıqda universitet laboratoriyasındakı printer növbəsini modelləşdirəcəksiniz.

Queue prinsipi:

First In → First Out (FIFO)

ilk daxil olan çap işi ilk emal edilir.

**Yaradılacaq siniflər**

***PrintJobNode***

Saxlanılacaq məlumatlar:

Job ID

Student Name

Number of Pages

Pointer to Next Job

***PrintQueue***

Aşağıdakı funksiyalar dəstəklənməlidir:

enqueue()

dequeue()

peek()

displayQueue()

countJobs()

Proqramın işləmə ssenarisi

İstifadəçi aşağıdakı seçimləri etməlidir:

1. Add Print Job

2. Print Next Job

3. View Next Job

4. Display Queue

5. Count Jobs

6. Exit

**Nümunə**

Çap növbəsinə aşağıdakı işlər əlavə edilir:

Ali - 5 pages

Leyla - 10 pages

Nigar - 3 pages

Queue görünüşü:

FRONT

Ali

Leyla

Nigar

REAR

İstifadəçi "Print Next Job" seçdikdə:

Ali printed successfully.

Queue belə dəyişir:

FRONT

Leyla

Nigar

REAR

**Tələbə öyrənəcək**

- Queue ADT
- Front və Rear göstəriciləri

- Enqueue və Dequeue əməliyyatları
- FIFO prinsipi
- Növbə əsaslı sistemlərin işləmə məntiqi

---

**Əlavə qeyd (Bütün tapşırıqlar üçün)**

**C++ istifadə edən tələbələr:**

std::list

std::stack

std::queue

std::deque

istifadə edə bilməzlər.

**Python istifadə edən tələbələr:**

collections.deque

queue.Queue

queue.LifoQueue

queue.PriorityQueue

istifadə edə bilməzlər.

Tələbələr bütün strukturları:

- öz Node sinifləri ilə,
- pointer/reference əlaqələri ilə,
- sıfırdan implementasiya etməlidirlər.

Bu yanaşma Weiss və Goodrich kitablarında vurğulanan əsas məqsədə, yəni tələbənin verilənlər strukturunun daxilində nə baş verdiyini başa düşməsinə xidmət edir.